

Universidad Veracruzana

Región Poza Rica - Tuxpan

Facultad de Ingeniería en Electrónica y Comunicaciones

Ingeniería en Tecnologías Computacionales

Emulación del procesador RISC-V (RV32) en lenguaje Java

Experiencia educativa: Microprocesadores y Microcontroladores

Presenta(n):

Luis Jaziel Basilio Cruz zS24006908

Juan Daniel Benitez Ordoñez zs24006915

César Uriel Basilio Peres zs2400692

20 de Octubre de 2026

"Lis de Veracruz: Arte, Ciencia, Luz"

Introducción

Tres razones para elegir RISC-V



Abierto

Especificación libre y sin regalías, mantenida por una organización sin fines de lucro.



Simple y modular

Una base de 40 instrucciones (RV32I) y extensiones opcionales: M, A, F, D y C.



Vigente

Unos 10 mil millones de núcleos estimados en 2022 y más de 4,500 miembros en 2025.

RISC-V frente a x86 y Arm

	x86	Arm	RISC-V
Licencia	Propietaria (Intel, AMD)	Licencia comercial de Arm	Estándar abierto, sin regalías
Longitud de instrucción	Variable, de 1 a 15 bytes.	Fija de 32 bits (AArch64)	Fija de 32 bits en la base.
Especificación	Manuales del fabricante	Documentación del fabricante	Pública y con comunidad abierta
¿Se implementa completa en un curso?	No es práctico	Requiere licencia	Sí, sin pedir permiso.

Última fila: valoración propia con fines didácticos.

¿Qué nos propusimos?

Entender un procesador viéndolo trabajar, no solo leyendo sobre él.

OBJETIVO GENERAL

Construir un emulador académico de RISC-V RV32, , que ejecute programas en ensamblador y muestre cómo trabaja el procesador por dentro.

Simular con fidelidad

Camino de datos, unidad de control y ALU como en el hardware.

Aprender ensamblando

Editor y ensamblador con etiquetas y pseudo-instrucciones.

Hacerlo visible

Registros, RAM y traza de cada etapa en pantalla.

Demostrar corrección

Pruebas automáticas con resultado esperado y obtenido.

Breve historia de RISC-V

1981 Berkeley crea RISC-I y RISC-II: nace la idea de un conjunto de instrucciones reducido.

2010 En verano, Asanović, Patterson y sus estudiantes inician RISC-V, la quinta generación RISC.

2011 Se publica la primera especificación con licencias abiertas de tipo BSD.

2015 Se funda la RISC-V Foundation con 36 miembros fundadores.

2019 Se ratifica la ISA base (versión 20191213).

2020 Se constituye RISC-V International en Suiza, una organización neutral y global.

De un proyecto a un estándar global

2010
Proyecto de verano en UC Berkeley

2011
Se publica la especificación con licencias abiertas

2015
Nace la RISC-V Foundation, con 36 miembros fundadores

2019
Se ratifica el ISA de usuario (versión 20191213)

2020
RISC-V International se constituye en Suiza

2022
Se estiman 10 mil millones de núcleos en el mercado.

2025
Más de 4,500 miembros

Fuentes: riscv.org/about · Wikipedia (RISC-V) · EE News Europe, julio de 2022

Importancia de RISC-V en el mundo actual

≈10 mil millones
de núcleos RISC-V (estimación, 2022)

+4 500
miembros de RISC-V International (2025)

Dónde se nota

- **Microcontroladores e IoT:** serie ESP32-C de Espressif.
- **Almacenamiento:** Western Digital adoptó RISC-V.
- **Industria:** MIPS pasó de su propia ISA a RISC-V.
- **Software:** Android incorporó soporte oficial.
- **Educación e investigación:** ISA abierta y extensible.

Fuentes: riscv.org; EE News Europe (2022); Wikipedia

Seis capas, cada una con un trabajo

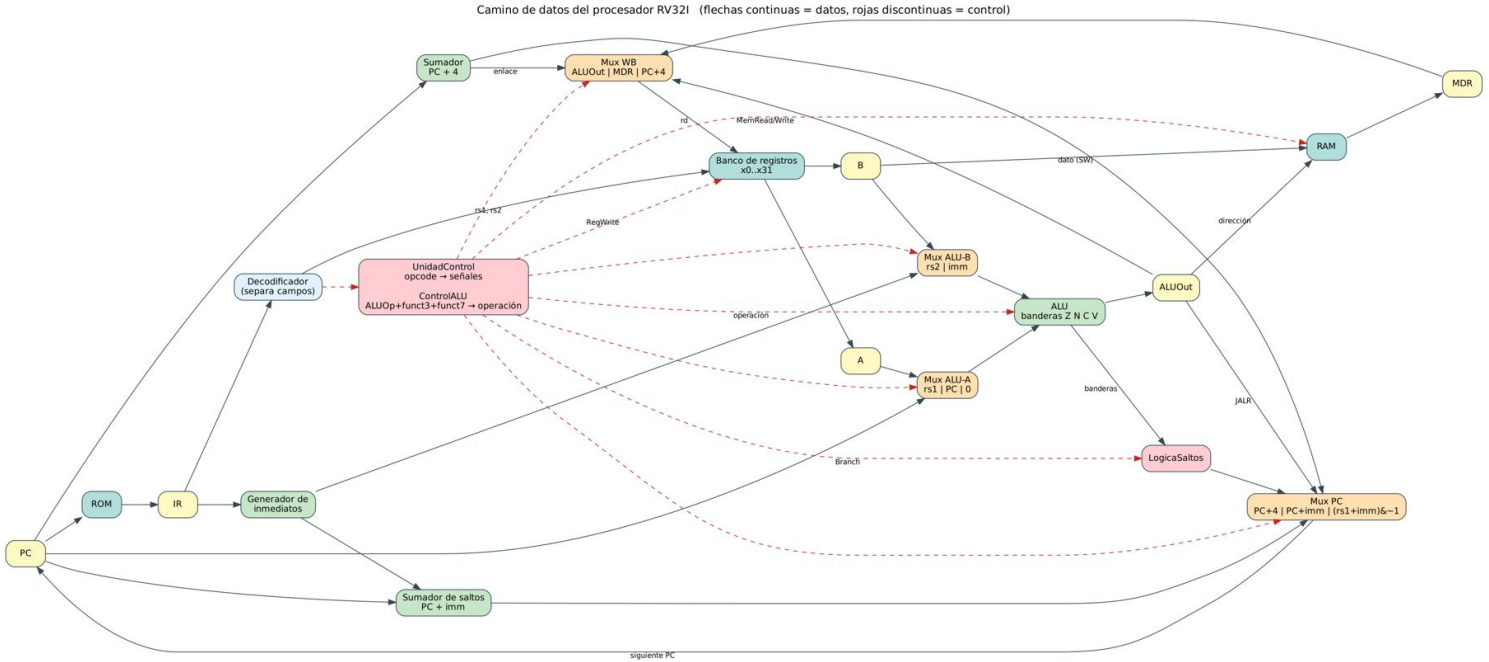
1	**Presentación (Swing)**	Main · ModeloFilas · RenderizadorFilas
2	**Aplicación**	CargadorDePrograma · Ensamblador · Microprocesador
3	**Procesador**	RutaDeDatos · UnidadControl · ControlALU · ALU · Decodificador
4	**Memoria y registros**	UnidadAlmacenamiento · RAM · ROM · BancoRegistros · ContadorPrograma
5	**Definición de la ISA**	TablaISA · DefinicionInstruccion · Instruccion
6	**Pruebas y utilidades**	AutoVerificacion · PlanPruebas · PruebasGUI · MainConsola

Una capa solo conoce a las de abajo.

Por eso el mismo procesador corre desde la consola sin tocar una línea de Swing.

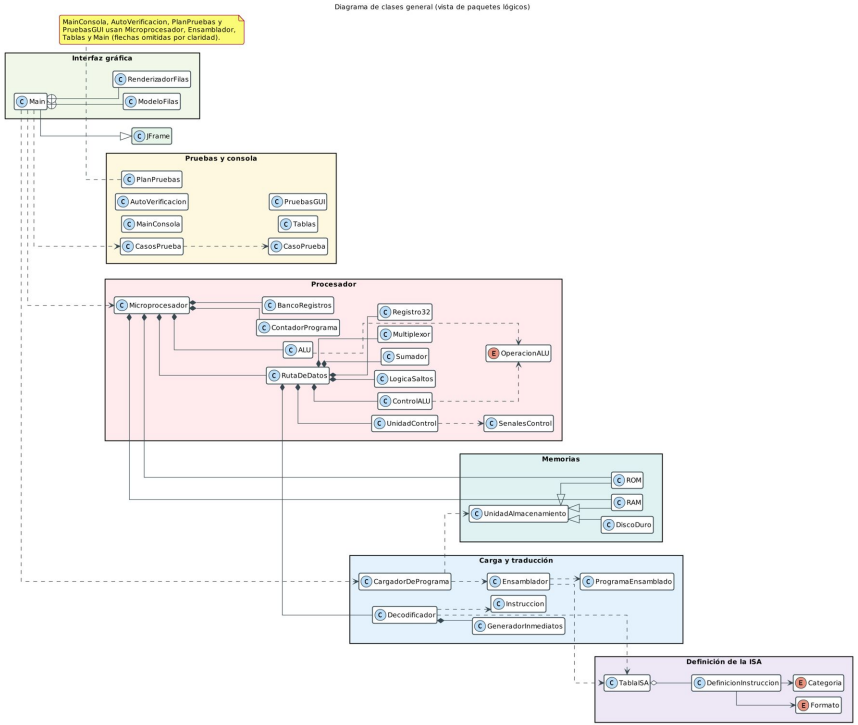
`33 archivos .java`

Arquitectura del procesador RISC-V



Camino de datos multiciclo: Fetch → Decode → Execute → Memory → Write-back

Diagrama de clases final del proyecto



- 35 archivos .java sin librerías externas.
- Seis paquetes lógicos: GUI, pruebas, carga, ISA, procesador y memorias.
- Herencia en memorias, polimorfismo en ROM y composición en Microprocesador.
- Fachada, MVC, Observer y diseño dirigido por datos (TablaISA).

La ISA es el contrato del procesador.

Programa
lo que escribe el usuario

Compilador / ensamblador
lo traduce a instrucciones

ISA: el contrato
RV32I define qué significa cada instrucción

Microarquitectura
cómo se construye: ALU, registros, control

Circuitos
compuertas y transistores

Quien controla la ISA controla qué se puede construir sobre ella.

- ISA propietaria: licencias, regalías y permisos.
- ISA abierta: cualquiera puede estudiarla, implementarla y extenderla.
- En un curso, lo ideal es poder implementar la ISA completa.

Procesador a emular: RISC-V RV32I

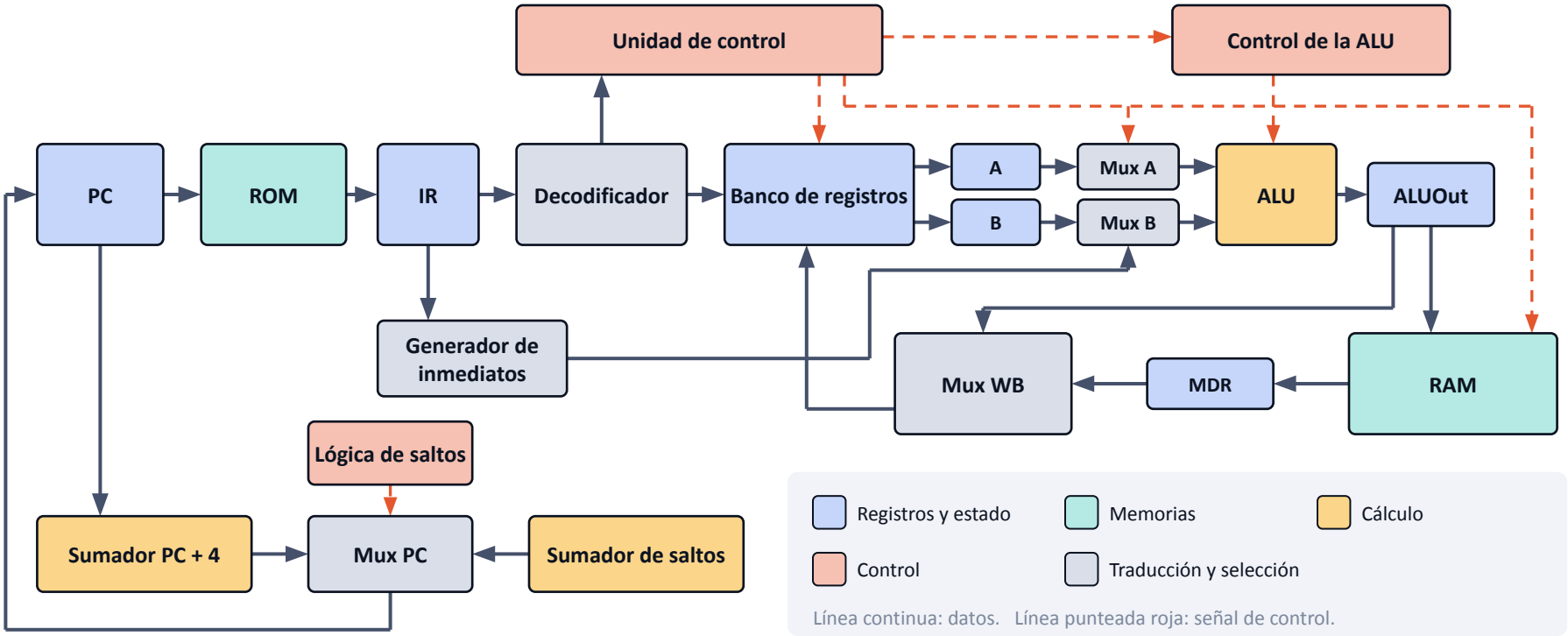
Categoría	Instrucciones	N.º
Aritmético-lógicas	add, sub, sll, slt, sltu, xor, srl, sra, or, and	10
Extensión M	mul, div, rem	3
Con inmediato	addi, slti, sltiu, xori, ori, andi, slli, srli, srai	9
Cargas	lb, lh, lw, lbu, lhu	5
Almacenamientos	sb, sh, sw	3
Saltos condicionales	beq, bne, blt, bge, bltu, bgeu	6
Saltos y constantes	jal, jalr, lui, auipc	4
Sistema	ecall, ebreak	2

Total: 42 instrucciones, más pseudoinstrucciones del ensamblador (li, mv, j, bgt, ble…)

Paso 1. Arquitectura del sistema de almacenamiento de datos



El mapa completo



PC

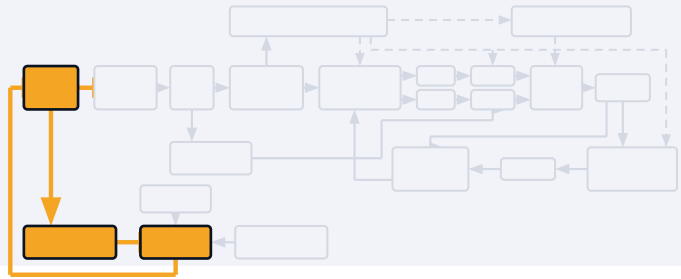
Es el dedo que va señalando el renglón de la receta que toca leer.

RECIBE	La dirección de la siguiente instrucción, que elige el Mux PC.
HACE	Guarda la dirección actual. Normalmente avanza de 4 en 4 bytes.
ENTREGA	La dirección que se usa para leer la ROM.

EJEMPLO REAL

`PC: 0 → 4 → 8 → 12 → 16 → 20 → 28 (salto)`

Dónde está en el procesador



Valores del PC en el programa de ejemplo

0	4	8	12	16	20	28

De 4 en 4, hasta que el beq salta de 20 a 28.

EN EL CÓDIGO `ContadorPrograma`

Contador de programa (PC)

- Registro de 32 bits con la dirección de byte de la instrucción actual.
- Inicia en 0 y suma 4 en cada instrucción, porque todas miden 4 bytes.
- Los saltos (branch, JAL y JALR) lo cargan con otra dirección.
- La instrucción actual está en ROM[PC / 4].

```
INCREMENTO = 4
pc = 0           // reinicio
pc = pc + 4      // secuencial
pc = destino     // salto
indiceROM = pc / 4
```

PC = 0	**PC = 4**	**PC = 8**	**PC = 12**
ROM[0]	ROM[1]	ROM[2]	ROM[3]

ROM e IR

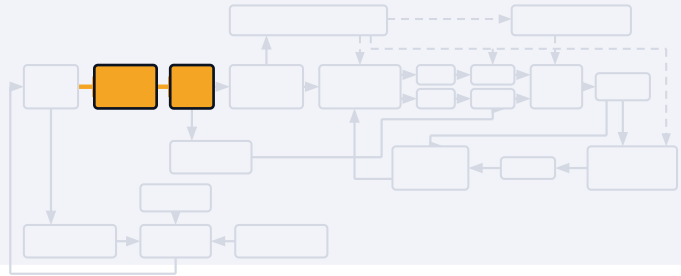
La ROM es el libro de recetas. El IR es la tarjeta donde copias el renglón que estás haciendo.

RECIBE	La dirección del PC (la ROM usa PC ÷ 4 para elegir la palabra).
HACE	La ROM solo se lee. El IR guarda la instrucción mientras se ejecuta.
ENTREGA	Una palabra de 32 bits con la instrucción.

EJEMPLO REAL

`PC = 8 → ROM[2] = 0x006283B3 → IR`

Dónde está en el procesador

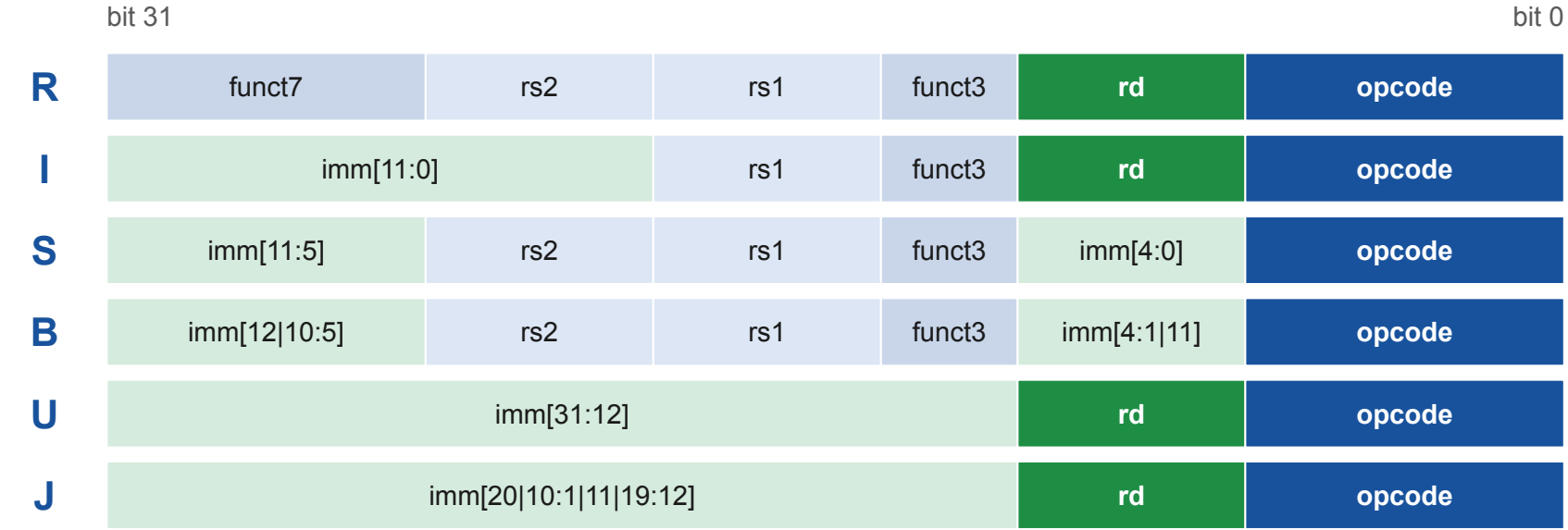


ROM[0]	0x00C00293
ROM[1]	0x01E00313
ROM[2]	0x006283B3 → IR
ROM[3]	0x00702023

La palabra 2 pasa al IR y queda ahí mientras se ejecuta.

EN EL CÓDIGO `ROM · Registro32`

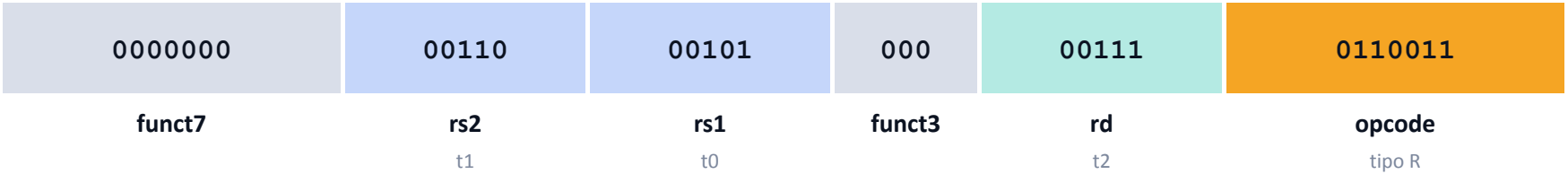
Análisis y decodificación de instrucciones



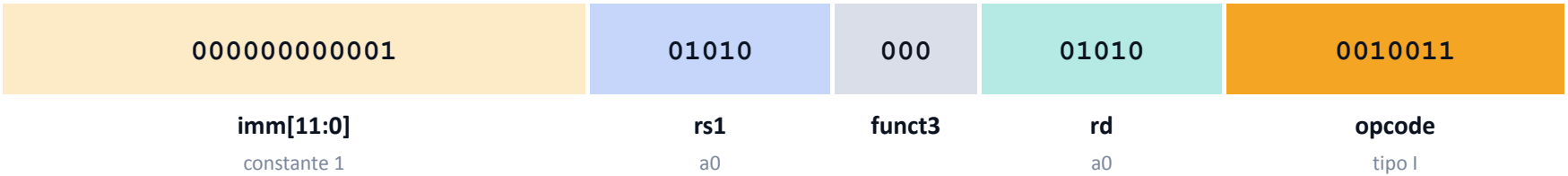
El Decodificador aísla cada campo con desplazamientos y máscaras; GeneradorInmediatos reconstruye el inmediato con extensión de signo y TablaISA (opcode + funct3 + funct7) identifica la instrucción.

Una instrucción, 32 bits

`add t2, t0, t1` tipo R `0x006283B3`



`addi a0, a0, 1` tipo I `0x00150513`



Seis formatos del mismo ancho: R, I, S, B, U y J. El decodificador separa los campos con desplazamientos y máscaras.

Decodificador e inmediatos: el traductor

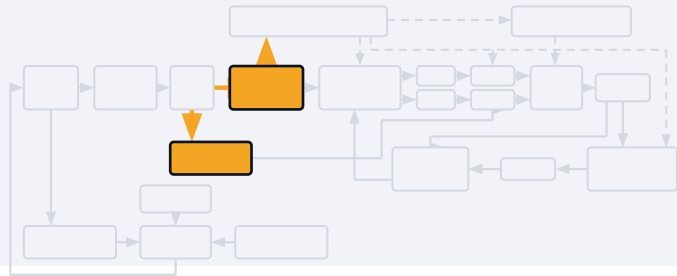
Es quien lee la tarjeta y separa qué operación pide, con qué ingredientes y cuánta cantidad.

RECIBE	Los 32 bits de la instrucción desde el IR.
HACE	Separa los campos: opcode, registros y funct. El generador arma la constante (inmediato).
ENTREGA	Campos listos, el inmediato con signo y el opcode para la unidad de control.

EJEMPLO REAL

`add t2,t0,t1 → rd=x7  rs1=x5  rs2=x6  funct7=0000000`

Dónde está en el procesador



add t2, t0, t1 separado en campos

0000000	00110	00101	000	00111	0110011
funct7	rs2	rs1	f3	rd	opcode

Inmediato: la constante que viaja dentro de la instrucción. En beq t2,t3,fin vale 8.

EN EL CÓDIGO **Decodificador · GeneradorInmediatos**

Unidad de Control



Unidad de control: el director

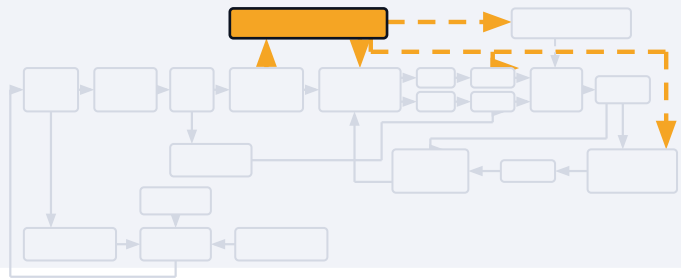
Es el director de orquesta: no toca nada, pero dice a cada pieza qué hacer en este momento.

RECIBE	El opcode, los 7 bits que dicen la familia de la instrucción.
HACE	Consulta una tabla fija y enciende o apaga señales de control.
ENTREGA	RegWrite, MemRead, MemWrite, Branch, Jump y los selectores de los muxes.

EJEMPLO REAL

`add → RegWrite=1  ALUSrcB=rs2  ALUOp=10  MemRead=0`

Dónde está en el procesador



	RegWrite	MemRead	MemWrite	Branch
add	1	0	0	0
lw	1	1	0	0
sw	0	0	1	0
beq	0	0	0	1

El opcode manda; las señales obedecen.

EN EL CÓDIGO `UnidadControl · SenalesControl`

Banco de registros

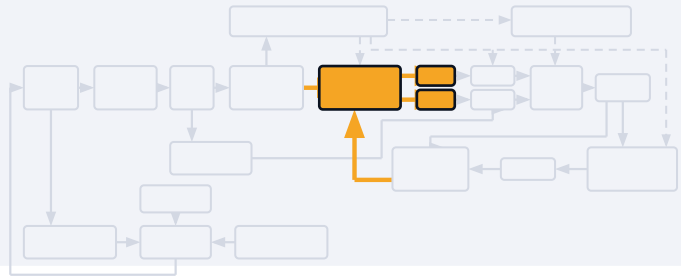
Son 32 cajones pequeños junto a la estufa, con lo que estás usando ahora mismo.

RECIBE	Dos números de registro para leer y uno para escribir, con su dato.
HACE	Entrega dos valores a la vez. El cajón x0 siempre vale 0.
ENTREGA	Los valores de rs1 y rs2, que se copian en A y B.

EJEMPLO REAL

`add t2,t0,t1 → lee x5 = 12 y x6 = 30; luego x7 ← 42`

Dónde está en el procesador



x0 **zero**	x1 **ra**	x2 **sp**	x3 **gp**	x4 **tp**	x5 **t0**	x6 **t1**	x7 **t2**
x8 **s0**	x9 **s1**	x10 **a0**	x11 **a1**	x12 **a2**	x13 **a3**	x14 **a4**	x15 **a5**
x16 **a6**	x17 **a7**	x18 **s2**	x19 **s3**	x20 **s4**	x21 **s5**	x22 **s6**	x23 **s7**
x24 **s8**	x25 **s9**	x26 **s10**	x27 **s11**	x28 **t3**	x29 **t4**	x30 **t5**	x31 **t6**

Gris: x0, siempre 0. Amarillo: t0, t1 y t2 del ejemplo.

EN EL CÓDIGO **BancoRegistros**

Paso 2. ALU

La ALU

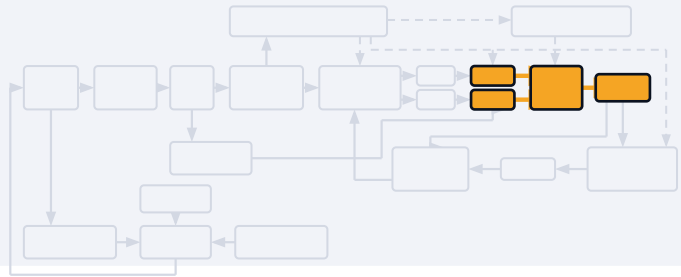
Recibe dos ingredientes, hace una operación y entrega un resultado.

RECIBE	Dos números de 32 bits y la operación que eligió ControlALU.
HACE	Suma, resta, AND, OR, XOR, desplazamientos, comparaciones, MUL, DIV y REM.
ENTREGA	El resultado (a ALUOut) y las cuatro banderas Z, N, C y V.

EJEMPLO REAL

```
ADD(12, 30) = 42   banderas Z=0 N=0 C=0 V=0
```

Dónde está en el procesador



Z	N	C	V
Cero	**Negativo**	**Acarreo**	**Desborde**
resultado = 0	resultado < 0	se pasó de 32 bits	error con signo

Operaciones: ADD SUB AND OR XOR SLL SRL SRA SLT SLTU MUL DIV REM

EN EL CÓDIGO **ALU · OperacionALU**

Control de la ALU: elige la operación

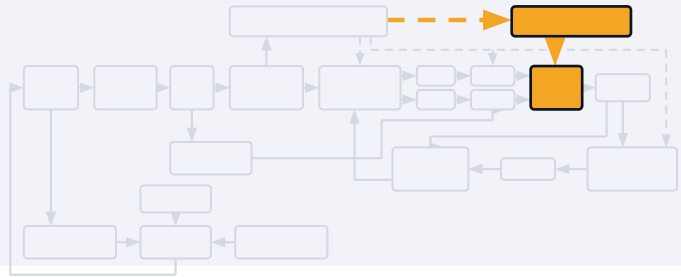
Es quien gira la perilla de la licuadora: decide qué operación hará la ALU.

RECIBE	ALUOp (2 bits) de la unidad de control, más funct3 y funct7 de la instrucción.
HACE	Combina las tres cosas con una tabla.
ENTREGA	El código de la operación: ADD, SUB, AND, SLT...

EJEMPLO REAL

`ALUOp=10 + funct3=000 + funct7=0000000 → ADD`

Dónde está en el procesador



ALUOp	Qué hace	Se usa en
00	Siempre suma	lw, sw (dirección)
01	Siempre resta	saltos (beq...)
10	Mira funct3 y funct7	add, sub, and...
11	Mira funct3	addi, andi...

EN EL CÓDIGO `ControlALU`

Multiplexores y sumadores: desvíos y sumas

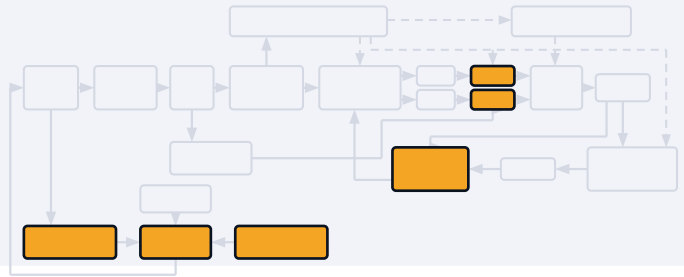
Los multiplexores son agujas de tren: escogen por qué vía pasa cada dato. Los sumadores solo suman.

RECIBE	Varios datos posibles y una señal de selección.
HACE	El mux deja pasar solo uno. Un sumador calcula PC + 4 o PC + inmediato.
ENTREGA	El dato elegido, o la suma.

EJEMPLO REAL

`Mux B elige: rs2 (add) o inmediato (addi, lw, sw)`

Dónde está en el procesador



Mux A	`rs1 | PC | 0`
Mux B	`rs2 | inmediato`
Mux WB	`ALUOut | MDR | PC+4`
Mux PC	`PC+4 | PC+inm. | JALR`

Sumadores: PC + 4 · PC + inmediato

EN EL CÓDIGO `Multiplexor · Sumador`

RAM

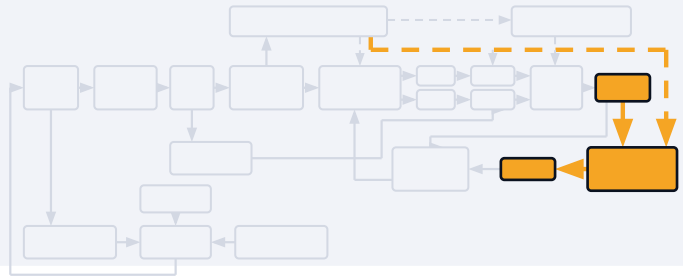
Es la despensa: grande, ordenada y más lenta de alcanzar que los cajones de la mesa.

RECIBE	Una dirección (la calcula la ALU) y, si es un sw, el dato a guardar.
HACE	Lee o escribe un byte, media palabra o una palabra, según la instrucción.
ENTREGA	El dato leído, que se guarda en MDR.

EJEMPLO REAL

`sw t2,0(zero) → RAM[0] ← 42` `lw t3,0(zero) → MDR = 42`

Dónde está en el procesador



La palabra 42 (0x0000002A) en la RAM, byte a byte

2A	00	00	00
byte 0	byte 1	byte 2	byte 3

Little-endian: el byte menos significativo va primero.

EN EL CÓDIGO **RAM**

Registros internos: las manos libres

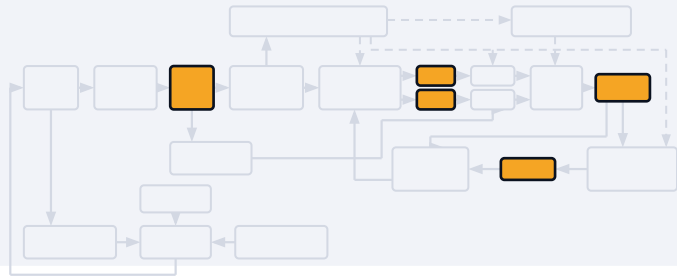
Son las manos del cocinero: sostienen un ingrediente mientras la otra mano trabaja.

A y B	Guardan los dos valores leídos del banco de registros.
ALUOUT	Guarda el resultado de la ALU hasta la etapa siguiente.
MDR Y IR	MDR guarda el dato leído de la RAM. IR guarda la instrucción.

EJEMPLO REAL

`add t2,t0,t1: A = 12, B = 30, ALUOut = 42; al final t2 ← ALUOut`

Dónde está en el procesador

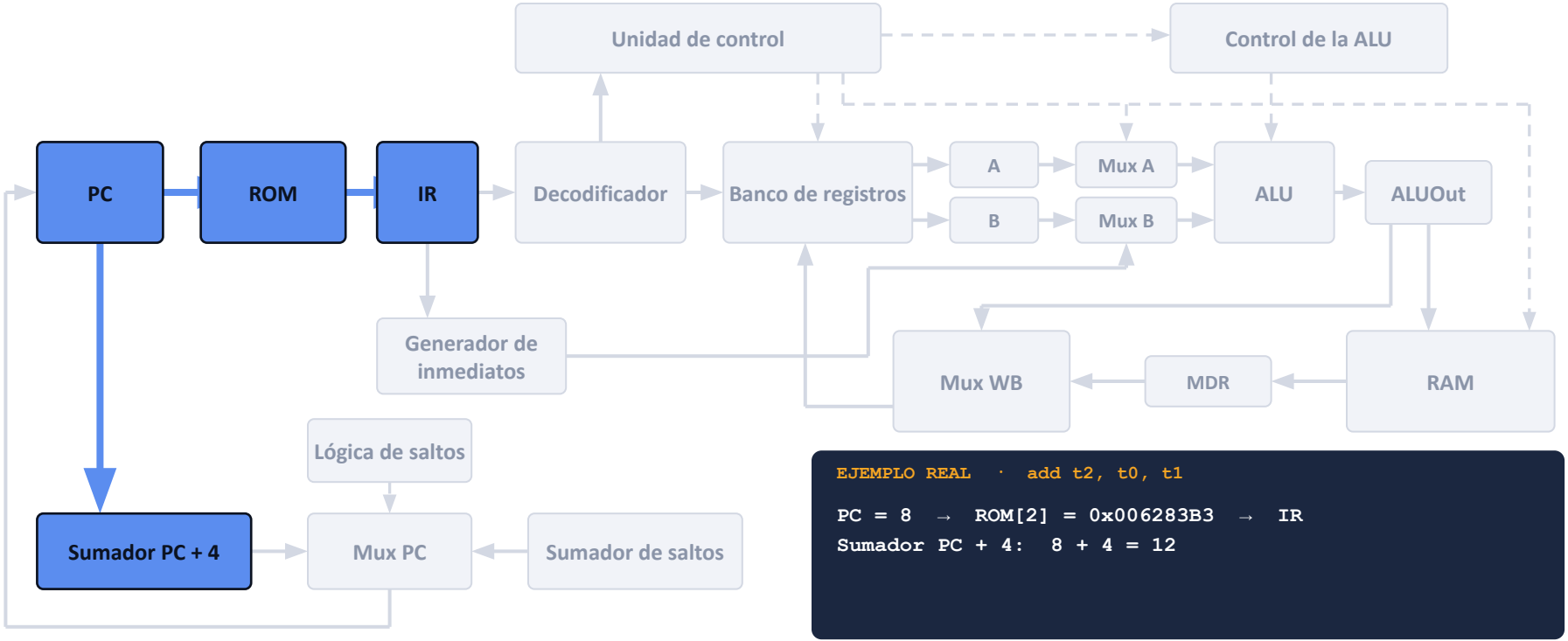


A	= 12
B	= 30
ALUOut	= 42
MDR	= (sin uso)

EN EL CÓDIGO `Registro32`

Etapa 1 · Fetch: traer la instrucción

El PC señala una dirección de la ROM; la instrucción llega al IR y se calcula PC + 4.

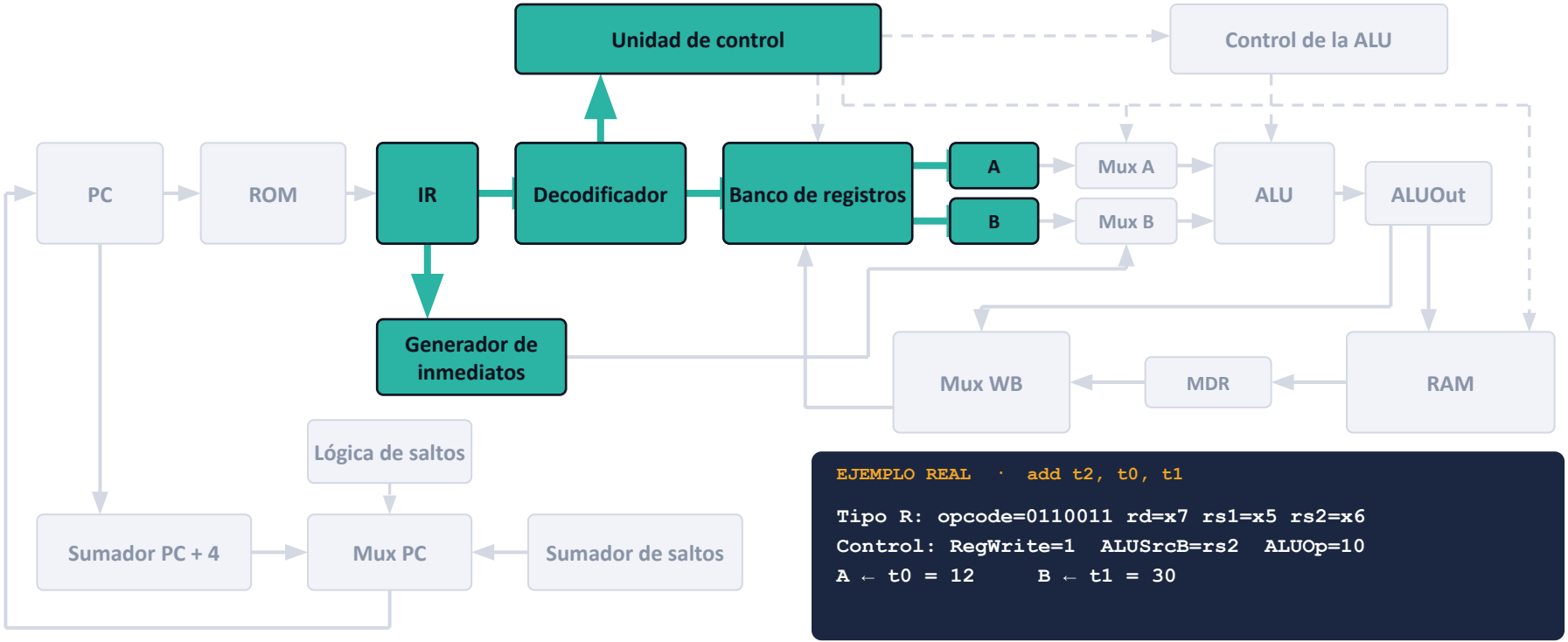


EJEMPLO REAL · add t2, t0, t1

PC = 8 → ROM[2] = 0x006283B3 → IR
Sumador PC + 4: 8 + 4 = 12

Etapa 2 · Decode: entender la instrucción

Se separan los campos, se generan las señales de control y se leen los registros A y B.

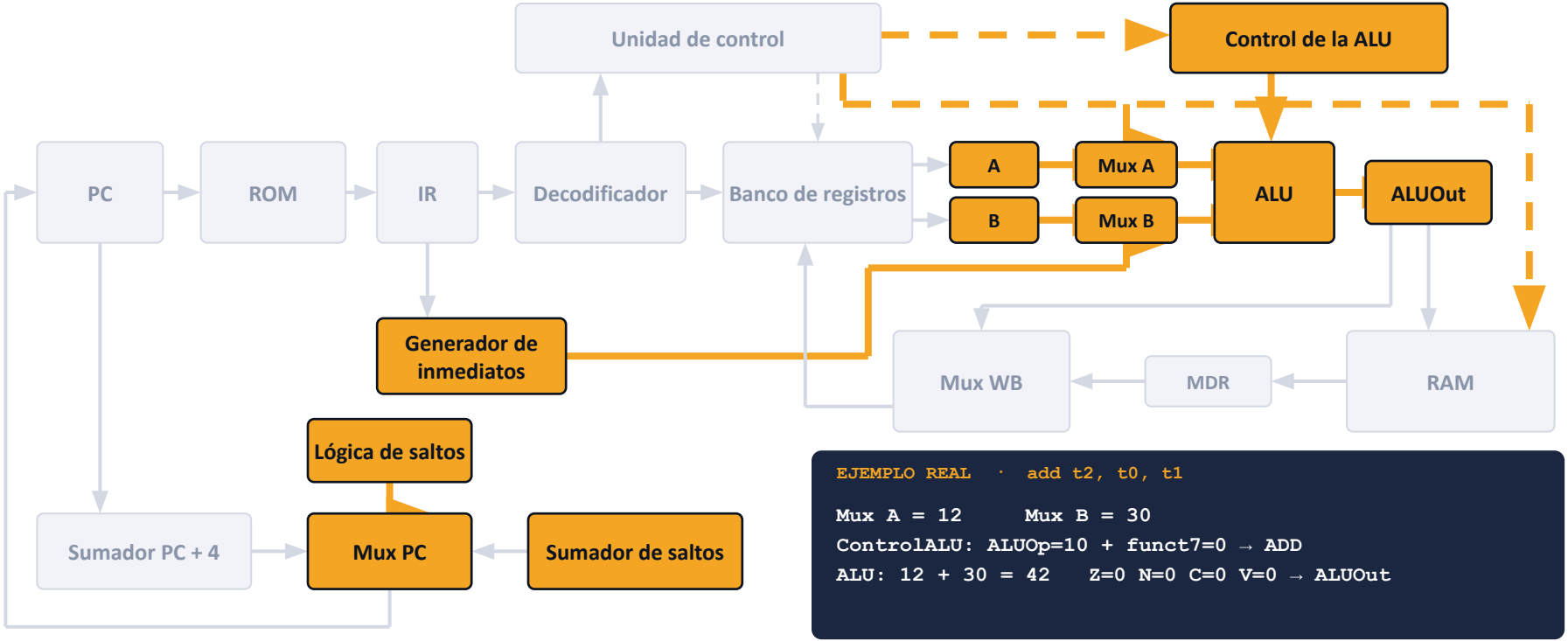


EJEMPLO REAL · add t2, t0, t1

```
Tipo R: opcode=0110011 rd=x7 rs1=x5 rs2=x6
Control: RegWrite=1  ALUSrcB=rs2  ALUOp=10
A ← t0 = 12      B ← t1 = 30
```

Etapa 3 · Execute: hacer el cálculo

Los muxes eligen operandos, ControlALU escoge la operación y la ALU calcula.

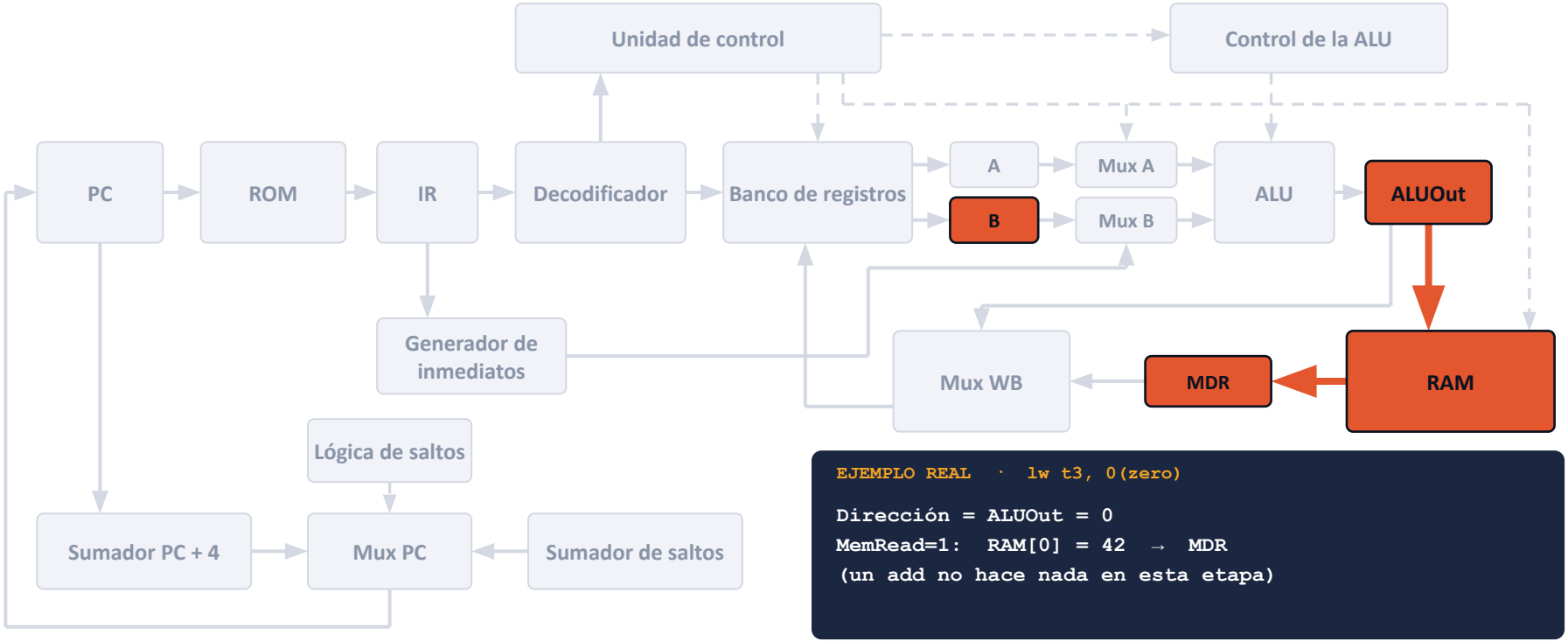



```
EJEMPLO REAL  ·  add t2, t0, t1

Mux A = 12      Mux B = 30
ControlALU: ALUOp=10 + funct7=0 → ADD
ALU: 12 + 30 = 42    Z=0 N=0 C=0 V=0 → ALUOut
```

Etapa 4 · Memory: leer o escribir la RAM

Solo las instrucciones de carga y almacenamiento usan la RAM. Las demás pasan de largo.

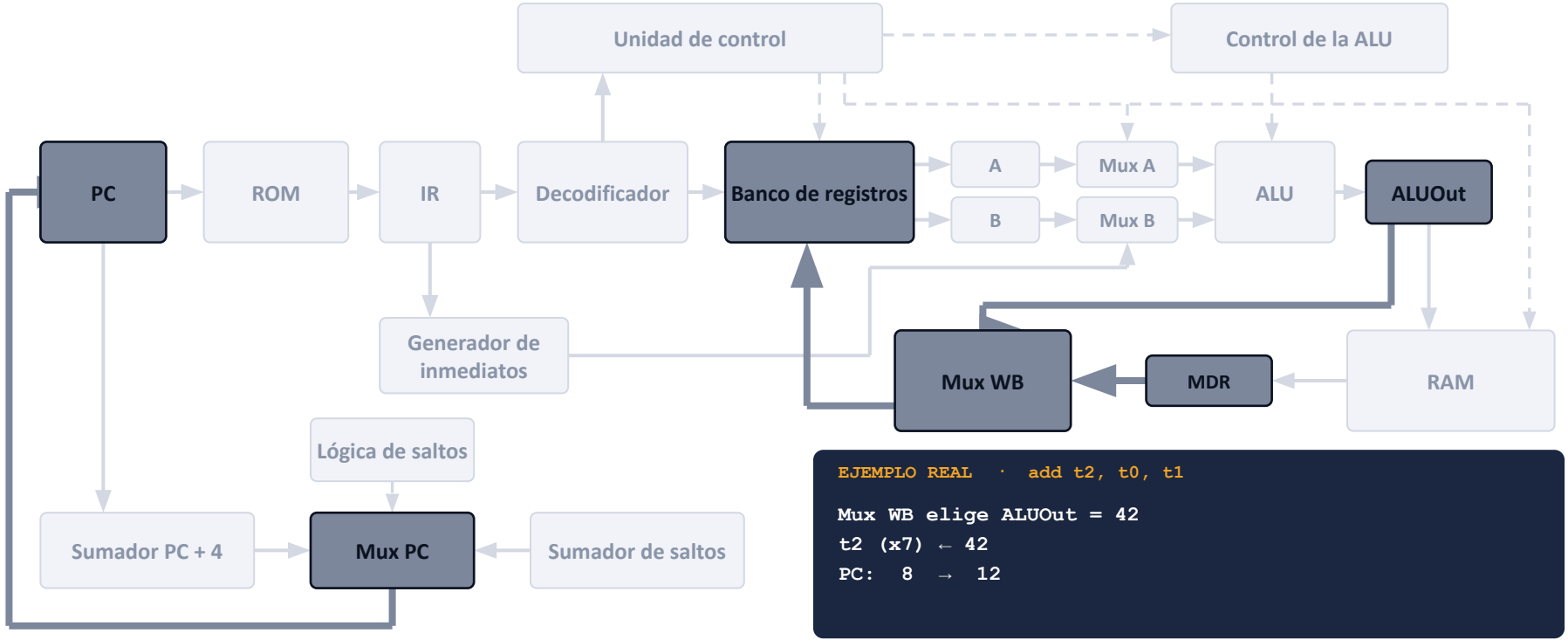



```
EJEMPLO REAL  ·  lw t3, 0(zero)

Dirección = ALUOut = 0
MemRead=1:  RAM[0] = 42  →  MDR
(un add no hace nada en esta etapa)
```

Etapa 5 · Write-back: guardar y avanzar

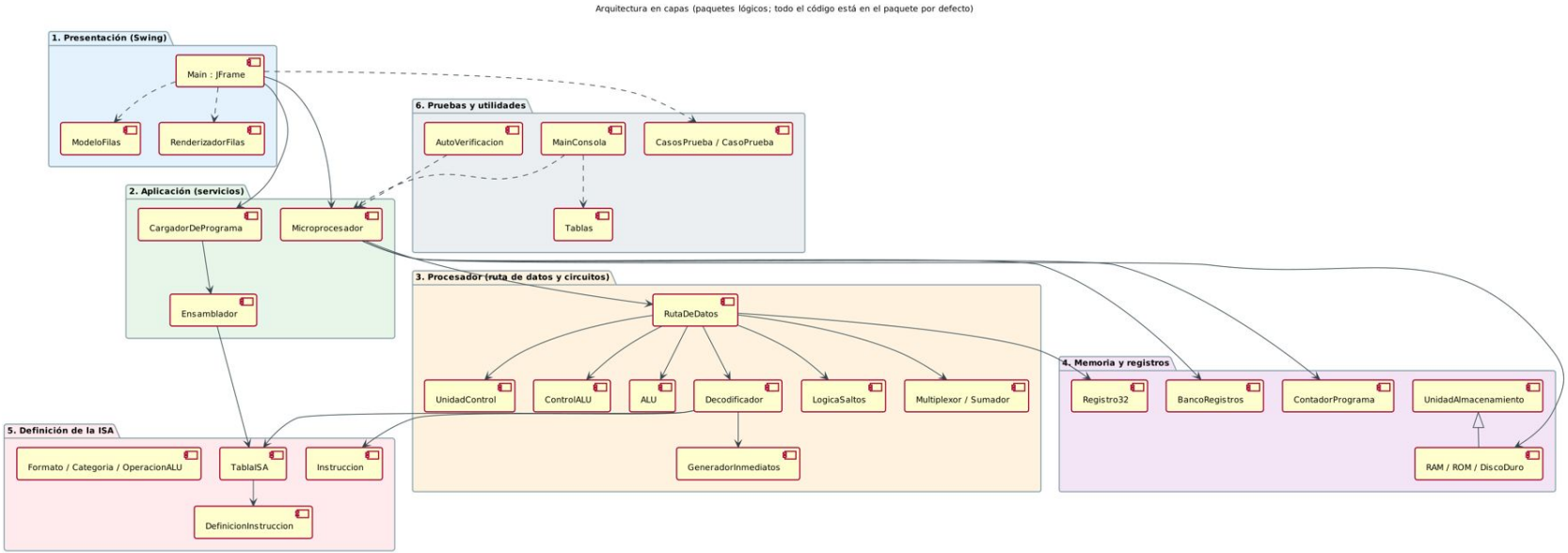
El Mux WB elige qué se escribe en el registro destino y el PC toma su nuevo valor.




```
EJEMPLO REAL  ·  add t2, t0, t1

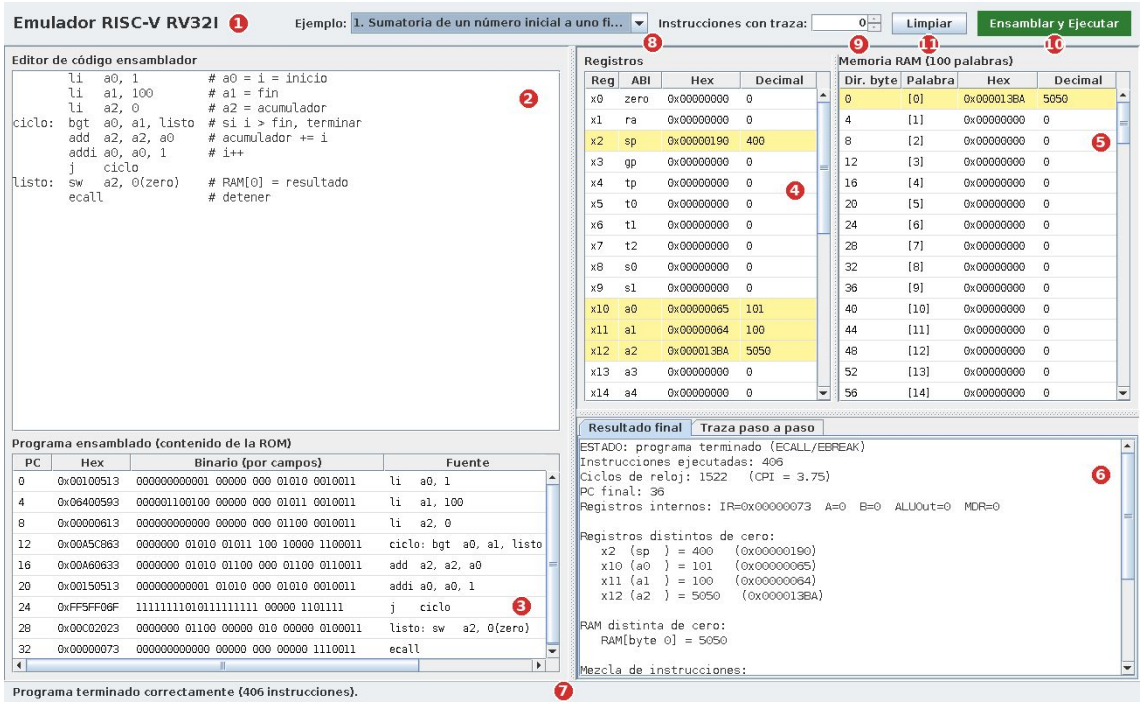
Mux WB elige ALUOut = 42
t2 (x7) ← 42
PC:  8  →  12
```

Arquitectura implementada del procesador RISC-V



Seis capas lógicas: interfaz gráfica, pruebas, carga y traducción, definición de la ISA, procesador y memorias.

La interfaz muestra la máquina por dentro.



1. Título
2. Editor de ensamblador
3. Programa ensamblado (ROM)
4. Registros x0 a x31
5. Memoria RAM
6. Resultado final y traza
7. Barra de estado
8. Selector de ejemplos
9. Instrucciones con traza
10. Ensamblar y Ejecutar
11. Limpiar

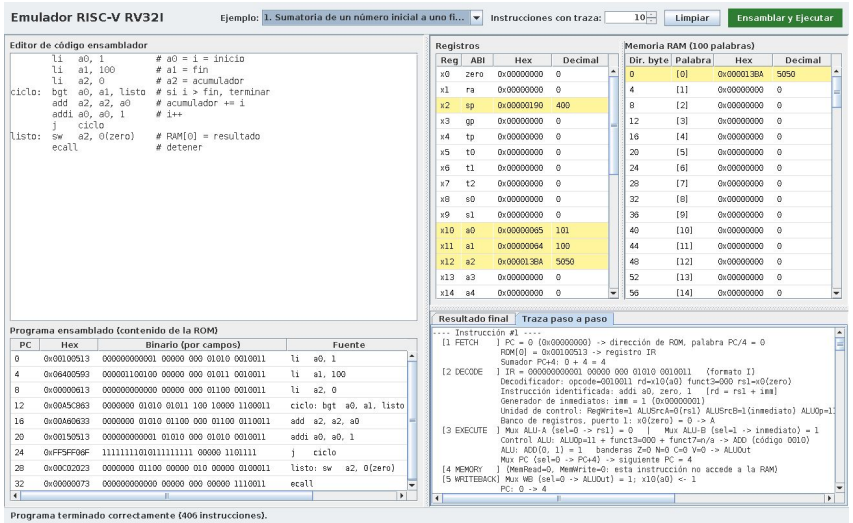
Paso 3. Pruebas técnicas al simulador del procesador RISC-V



P1. Sumatoria desde un número inicial a uno final

Suma 1 + 2 + … + 100 con un ciclo: 100 × 101 / 2 = 5050.

```
ciclo: bgt   a0, a1, listo
       add   a2, a2, a0
       addi  a0, a0, 1
       j     ciclo
listo: sw    a2, 0(zero)
```



RAM[0] = 5050

Coincide con el valor esperado

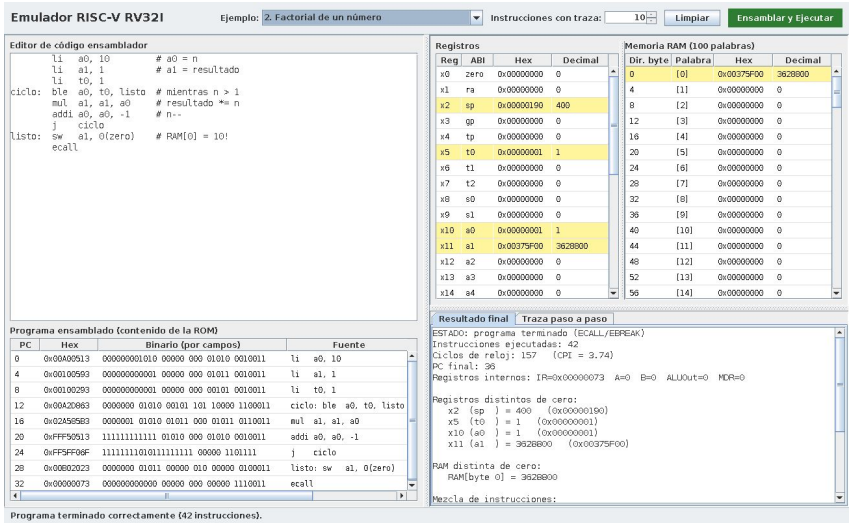
406 instrucciones · 1 522 ciclos · CPI 3.75

9 palabras de programa

P2. Factorial de un número

Calcula 10! con MUL (extensión M) en un ciclo.

```
ciclo: ble  a0, t0, listo
       mul  a1, a1, a0
       addi a0, a0, -1
       j    ciclo
listo: sw   a1, 0(zero)
```



RAM[0] = 3 628 800

Coincide con el valor esperado

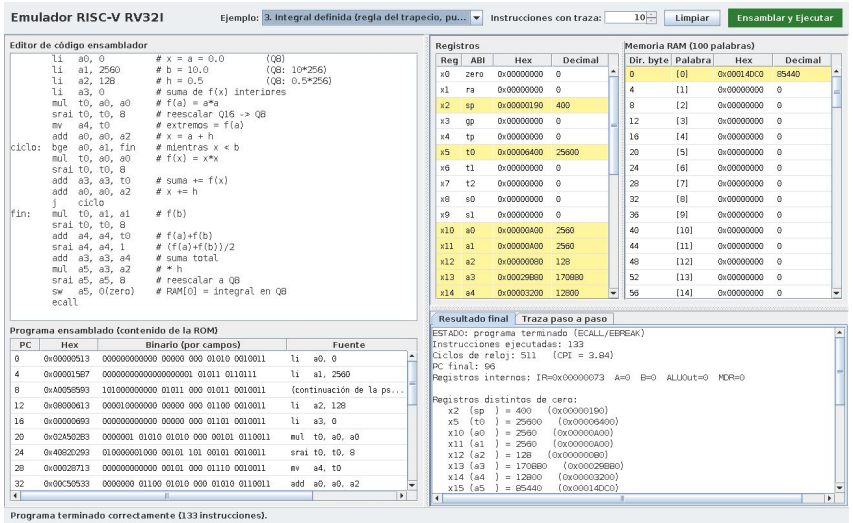
42 instrucciones · 157 ciclos · CPI 3.74

9 palabras de programa

P3. Integral definida desde un punto inicial a uno final

Integral de x^2 en [0, 10] por trapecio, h = 0.5, en punto fijo Q8 (real = entero / 256).

```
mul  t0, a0, a0   # f(x) = x*x
srai t0, t0, 8     # Q16 → Q8
add  a3, a3, t0   # suma
add  a0, a0, a2   # x += h
j    ciclo
```



RAM[0] = 85 440 → 333.75

Coincide con el valor esperado

133 instrucciones · 511 ciclos · CPI 3.84

24 palabras de programa

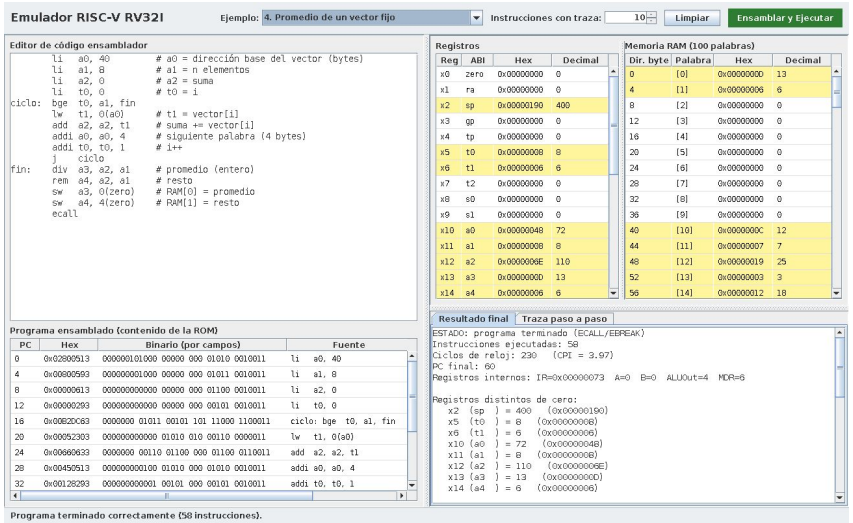
Paso 4. Pruebas de usuario final



P4. Promedio de un vector fijo de elementos

Promedio del vector {12, 7, 25, 3, 18, 9, 30, 6} guardado en la RAM.

```
lw   t1, 0(a0)     # vector[i]
add  a2, a2, t1    # suma
addi a0, a0, 4     # siguiente
div  a3, a2, a1    # promedio
rem  a4, a2, a1    # resto
```



RAM[0] = 13 y RAM[1] = 6

Coincide con el valor esperado

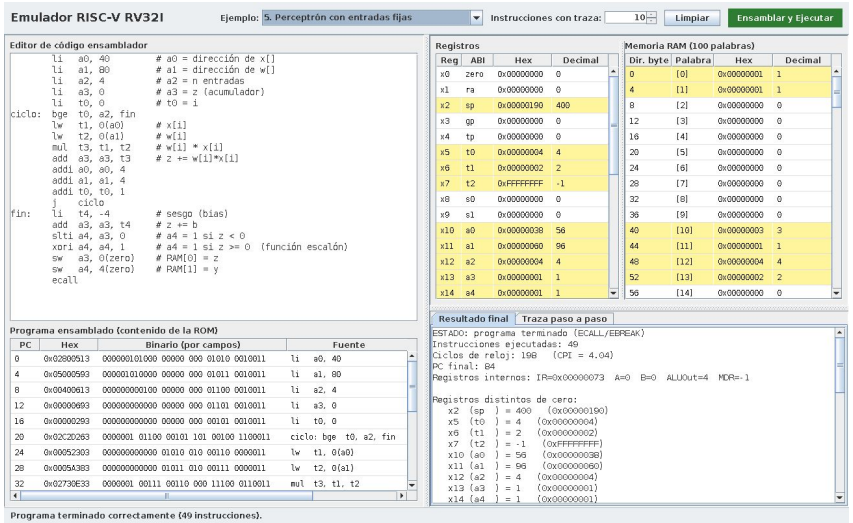
58 instrucciones · 230 ciclos · CPI 3.97

15 palabras de programa

P5. Cálculo de un perceptrón con entradas fijas

Perceptrón de 4 entradas: x = {3, 1, 4, 2}, w = {2, −3, 1, −1}, b = −4.

```
lw   t1, 0(a0)     # x[i]
lw   t2, 0(a1)     # w[i]
mul  t3, t1, t2
add  a3, a3, t3    # z += w*x
slti a4, a3, 0 ; xori a4, a4, 1
```



z = 1 → y = escalón(z) = 1

Coincide con el valor esperado

49 instrucciones · 198 ciclos · CPI 4.04

21 palabras de programa

Reflexiones finales

Conclusiones

1

Procesador funcional

RV32I + MUL/DIV/REM (42 instrucciones) con ciclo de 5 etapas y CPI de 3.74 a 4.04.

2

Resultados verificados

Los 5 programas coinciden con valores calculados aparte; 140/140 casos y 110/110 comprobaciones.

3

Buen diseño con POO

Del código estático de 2017 a 35 clases en 6 capas; una sola TablaISA sirve a ensamblador y decodificador.

4

RISC-V fue buena elección

Formatos regulares y especificación abierta facilitaron analizar, decodificar y probar cada instrucción.

Trabajo futuro

1. Segmentación (pipeline) con riesgos de datos y de control.
2. Más ISA: M completa (MULH, DIVU), C comprimida y F de punto flotante.
3. Directivas del ensamblador (.data, .word), FENCE y registros CSR.
4. Caché, interrupciones y periféricos mapeados en memoria.
5. Probar la interfaz en macOS (Linux virtual).

Alcance: qué incluye y qué no

Incluye

- 39 instrucciones de RV32I y MUL, DIV y REM
- Ensamblador con etiquetas y pseudo-instrucciones (li, mv, j, ret...)
- 32 registros, ROM y RAM de 100 palabras direccionables por byte
- Interfaz Swing y versión de consola

No incluye

- FENCE, CSR, coma flotante, MULH ni DIVU
- Directivas de datos (.data, .word)
- Programas de más de 100 instrucciones
- Pipeline: el modelo es de ciclo múltiple.



Universidad Veracruzana

Región Poza Rica - Tuxpan

Facultad de Ingeniería en Electrónica y Comunicaciones

Ingeniería en Tecnologías Computacionales

Emulación del Procesador RISC-V (RV32I) en lenguaje Java

Experiencia educativa: Microprocesadores y Microcontroladores

Presenta (n):

Luis Jaziel Basilio Cruz zS24006908

Juan Daniel Benitez Ordoñez zs24006915

César Uriel Basilio Peres zs24006925

20 de Octubre de 2026

"Lis de Veracruz: Arte, Ciencia, Luz"